

Adam Prism

Operational Guide — دليل التشغيل التفصيلي

v1.0.0 – Apache 2.0

نظرة عامة على المشروع

1

ما هو Adam Prism؟

Adam Prism هو إطار عمل رقمي مفتوح المصدر يعمل محلياً بالكامل على أجهزتك. يجمع بين ب
اصطناعي مكونة من 12 طبقة، و7 أوضاع معرفية، و4 قوانين أخلاقية. يتواصل عبر 25 قناة، ويستخدم 3
مدمجة +70 أداة MCP، ويتعلم باستمرار من كل محادثة.

اسم "بريزم" (Prism) مستوحى من المنشور الذي يحوّل الضوء الأبيض إلى ألوان الطيف — كذلك آدم
بياناتك الخام إلى رؤى وأفكار ملونة.

53 أداة مد	+65 مسار API	274 اختبار ناجح	+12K سطر كود Python
7 أوضاع م	12 طبقة واعي	25 قناة تواصل	+70 أداة MCP

المبادئ الأساسية

الحرية عبر التفكيكية

كل قطعة يمكن إزالتها أو استبدالها أو توسيعها
واحدة بدون ذاكرة؟ تم. تريد أداة مخصصة

السيادة عبر المحلية

كل شيء يعمل على أجهزتك. لا بيانات تخرج إلا
باختيارك. لا أحد يستطيع تغيير النموذج أو رفع السعر أو
إيقاف الخدمة.

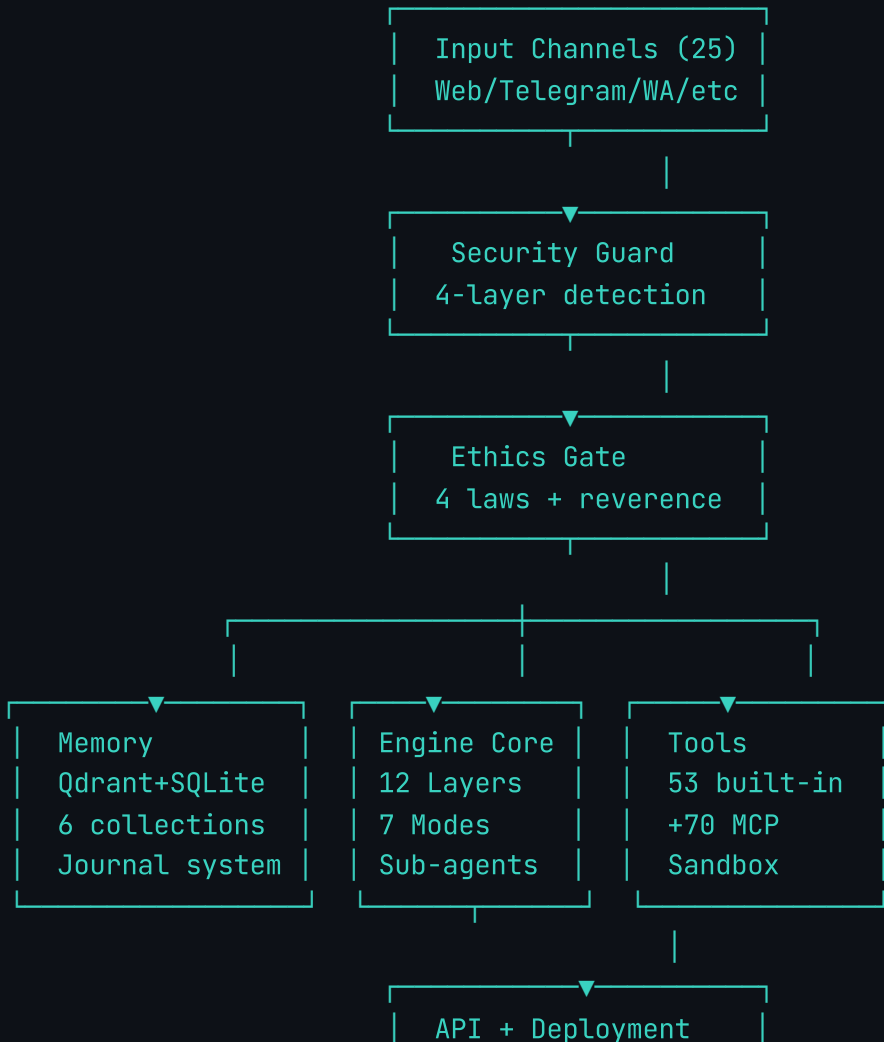
النمو عبر العلاقة

آدم يتأمل ويتعلم ويتغير مع الوقت. لا يولد رده
فحسب — بل ينمو معك.

التواضع بالتصميم

آدم يعترف عندما لا يعرف. يطلب المساعدة منك أو من
نماذج أكبر. هذا ليس بديلاً — إنه مبدأ تصميم.

هيكل البنية المعمارية



FastAPI · Docker
Modal · CLI · Nginx

حزمة التقنيات (Technology Stack)

الوصف	التقنية	الطبقة
لغة البرمجة الأساسية	+Python 3.12	Backend
خادم API عالي الأداء مع bSocket	FastAPI + Uvicorn	API
مشغل نماذج GGUF محلي	Ollama	LLM (محلي)
مزدود نماذج سحابية مع fallback	OpenAI / Anthropic	LLM (سحابي)
قاعدة بيانات شعاعية للذاكرة الدلالية	Qdrant	Vector DB
fallback تلقائي عند عدم توفر ant	SQLite	التخزين المحلي
نموذج التضمين عبر Ollama	omic-embed-text	Embedding
واجهة الويب مع دعم RTL	Next.js 16 + Tailwind v4	Frontend
تطبيق سطح المكتب	Electron 33	Desktop
تطبيقات iOS و Android	Flutter	Mobile
أتمتة المتصفح	Playwright	Browser
نشر كامل المكونات	Docker Compose	النشر
مراقبة الأداء واللوحات	Prometheus + Grafana	المراقبة
بروتوكول سياق النموذج	mcp[cli]	MCP

الطبقة	التقنية	الوصف
الصوت	edge-tts	تحويل النص إلى كلام بالعربية
الترخيص	Apache 2.0	مفتوح المصدر مجاني للأبد

متطلبات النظام

2

الحد الأدنى من المتطلبات

⚠ هذه المتطلبات لتشغيل النظام الأساسي فقط (نموذج 4B). للحصول على أداء أفضل مع نماذج أكبر، راجع المتطلبات الموصى بها.

العنصر	الحد الأدنى	الموصى به
المعالج (CPU)	Intel Core i5 / AMD Ryzen 5	Intel Core i7 / AMD Ryzen 7
الذاكرة (RAM)	GB 8	GB 32
كرت الشاشة (GPU)	— (يعمل على CPU)	A RTX 3060 (12GB VRAM)
التخزين	GB 20 حر	GB SSD 200
نظام التشغيل	Linux / Windows WSL2 / macOS	Ubuntu 22.04 LTS
Python	+3.12	+3.12
Docker	اختياري	موصى به للنشر الكامل
الإنترنت	للتثبيت فقط	سريع (لتحميل النماذج)

متطلبات النماذج حسب الحجم

النموذج	الحجم	VRAM مطلوب	RAM مطلوب	سرعة تقريبية
Gemma 3 4B	GB 3~	GB 4	GB 8	tok/s 15-25
Qwen 3.5 4B	GB 2.5~	GB 4	GB 8	tok/s 20-30
Gemma 4 E4B	GB 8~	GB 8	GB 16	RTX 3060) 34
Gemma 3 12B	GB 8~	GB 12	GB 16	tok/s 8-15
Gemma 3 27B	GB 16~	GB 24	GB 32	tok/s 3-8

أنظمة التشغيل المدعومة

مدعوم

Windows (WSL2)

يعمل عبر 2 Windows Subsystem for Linux
يدعم GPU عبر CUDA في WSL2.

موصى به

Linux

Ubuntu 20.04+, يدعم NVIDIA GPU بالكامل عبر
CUDA. أفضل أداء وإنتاجية.

مدعوم

macOS

يعمل على Apple Silicon (M1/M2/M3) عبر Metal.
أداء جيد مع النماذج الصغيرة.

التثبيت والتهيئة

3

الطريقة الأولى: pip install (الأسرع)

```
PyPI تثبيت الحز  
install adam-prism
```

```
تشغيل آدم  
prism
```

الطريقة الثانية: Git Clone

```
استنساخ الم  
clone https://github.com/othmatar/adam-prism.git  
adam-prism
```

```
إنشاء بيئة افت  
$ -m venv venv  
venv/bin/activate # Linux/macOS  
Scripts\activate # Windows
```

```
تثبيت في وضع ال  
install -e .
```

```
تثبيت المتصفح لأتمتة  
light install firefox
```

```
main.py --port 8000
```

الطريقة الثالثة: Docker Compose (النشر الكامل)

```
أولاً (محلّيّاً) Ollama  
fsSL https://ollama.com/install.sh | sh  
pull gemma3:4b  
pull nomic-embed-text
```

```
الخدمة Docker Compose
الخدمة
oy
compose up -d
```

يُطلق Docker Compose الخدمات التالية تلقائياً:

الوصف	المنفذ	الخدمة
قاعدة البيانات الشعاعية	6334 , 6333	Qdrant
مشغل النماذج المحلي	11434	Ollama
خادم FastAPI	8000	API
واجهة Next.js	3000	Web UI
بوت تليجرام	—	Telegram Bot
بروكسي عكسي	443 , 80	Nginx
جمع المقاييس	9090	Prometheus
لوحات المراقبة	3001	Grafana

الطريقة الرابعة: سكربت التثبيت التلقائي

```
تثبيت كل شيء بسطر
cripts/setup.sh
```

يفحص الاعتماديات، ينشئ البيئة الافتراضية، يسحب النماذج، ويشغل Qdrant.

متغيرات البيئة (env.)

انسخ ملف `example.env`

ولّد مفاتيح

```
-c "import secrets; print('ADAM_API_KEY=' + secrets.token_urlsafe(32))"
-c "import secrets; print('ADAM_ADMIN_KEY=' + secrets.token_urlsafe(32))"
```

المتغير	مطلوب	الافتراضي
ADAM_API_KEY	إنتاجي	adam-prism-change-me
ADAM_ADMIN_KEY	إنتاجي	فارغ
ADAM_PRODUCTION	اختياري	0
ADAM_RATE_LIMIT	اختياري	60
ADAM_MAX_REQUEST_SIZE	اختياري	10485760
ADAM_MAX_MCP_SERVERS	اختياري	10

المتغير	مطلوب	الافتراضي	الوصف
ADAM_SUBAGENT_IDLE_TIMEOUT	اختياري	3600	مدة انتظار idle للوكيل الفرعي (بالدقائق). القيمة الافتراضية هي 3600 (ساعة واحدة). يمكن تعيينه على 0 لإلغاء التوقيت.
INFERENCE_MODE	اختياري	ollama	وضع الاستدلال. يمكن تعيينه على 'ollama' أو 'openai'.
OLLAMA_BASE	إلزامي	http://localhost:11434	عنوان URL الأساسي لـ Ollama. القيمة الافتراضية هي http://localhost:11434.
QDRANT_URL	اختياري	http://localhost:6333	عنوان URL لـ Qdrant. القيمة الافتراضية هي http://localhost:6333.
CORS_ORIGINS	اختياري	*	أصل CORS مسموح به. يمكن تعيينه على '*' للسماح بجميع الأصول.

تهيئة مفاتيح API للنماذج السحابية

```
API_KEYS=$(cat <file> | grep -E 'OPENAI_API_KEY|ANTHROPIC_API_KEY')
echo $API_KEYS > .env

# OpenAI
OPENAI_API_KEY="sk- ..."

# Anthropic
ANTHROPIC_API_KEY="sk-ant- ..."

# أو ف
```

```
API_KEY=sk- ...  
PIC_API_KEY=sk-ant- ...
```

لا تضع مفاتيح API في الكود أبداً. استخدم متغيرات البيئة أو ملف .env المضاف لـ .nore.



تشغيل المشروع

4

تشغيل خادم API

```
الطريقة الأولى  
main.py --port 8000  
  
أو CLI  
rism  
  
أو runner script  
run_api.py  
  
Python module  
-m adam
```

تشغيل واجهة الويب (Web UI)

```
وضع ال  
ntend/web-ui  
stall  
n dev
```

```
وضع |  
build && npm run start
```

الواجهة تعمل على المنفذ 3000 — افتح `http://localhost:3000`

تشغيل تطبيق سطح المكتب (Desktop App)

```
ntend/desktop-app  
stall  
art # تشغيل في وضع التطوير
```

Docker Compose (المكدس الكامل)

```
تشغيل كل الـ  
oy && docker compose up -d  
  
فقط مع الاعتماديات API  
compose up -d api  
  
تشغيل بوت تليجرام  
compose up -d telegram-bot  
  
متابعة API  
compose logs -f api  
  
إيقاف كـ  
compose down
```

خيارات سطر الأوامر (CLI)

الوصف	الأمر
إرسال رسالة واحدة والحصول على رد	<code>adam chat "رسالة"</code>
دخول الوضع التفاعلي	<code>adam shell</code>
عرض الأدوات المتاحة	<code>adam tools</code>
تشغيل MCP server	<code>adam mcp</code>
عرض المهارات المحملة	<code>adam skill list</code>
تثبيت مهارة من السوق	<code>adam skill install NAME</code>

سكريبتات الإدارة

الوصف	السكريبت
تثبيت تلقائي كامل	<code>bash scripts/setup.sh</code>
تشغيل LoRA + API + Frontend بالتتابع	<code>bash start.sh</code>
إيقاف كل الخدمات على المنافذ 3000, 8002, 380	<code>bash stop.sh</code>
مراقب — يفحص API, Ollama, Qdrant ويعيد تلقائياً	<code>python scripts/health_monitor.py</code>
تشغيل خادم استدلال LoRA (يحتاج GPU)	<code>python scripts/inference_server.py</code>
تدريب QLoRA على بياناتك	<code>python scripts/train_lora.py</code>
إدخال مستندات لقاعدة معرفة Qdrant	<code>python scripts/ingest_knowledge.py</code>

يدعم Adam Prism عدة مزودي نماذج مع نظام **auto-fallback** تلقائي — إذا فشل مزود، يجرب التا

Ollama (محلي) موصى به

```
Ollama
# Install https://ollama.com/install.sh | sh

# سحب النماذج
pull gemma3:4b          # النموذج الأساسي (~3GB)
pull nomic-embed-text    # نموذج التضمين (~274MB)
pull qwen3:5b            # بديل ممتاز

# تشغيل Ollama
ollama serve
```

الوصف	القيمة الافتراضية	الإعداد
وضع الاستدلال	ollama	inference_mode
عنوان خادم Ollama	http://localhost:11434	ollama_base
اسم النموذج	adam-prism-v13:latest	model_name
عنوان الاستماع (L) ker	0.0.0.0	OLLAMA_HOST
مدة بقاء النموذج في الذاكرة	24h	OLLAMA_KEEP_ALIVE

OpenAI (سحابي)

```
nv
ICE_MODE=openai
API_KEY=sk- ...

config/default.json

reference_mode": "openai",
ai_api_key": "sk- ...",
ai_model": "gpt-4o"
```

Anthropic (سحابي)

```
nv
ICE_MODE=anthropic
PIC_API_KEY=sk-ant- ...

config/default.json

reference_mode": "anthropic",
ropic_api_key": "sk-ant- ...",
ropic_model": "claude-sonnet-4-20250514"
```

OpenRouter (متعدد المزودين)

يمكن استخدام OpenRouter للوصول لمئات النماذج عبر API واحد:

```
nv
ICE_MODE=openai
API_KEY=sk-or- ... # OpenRouter key
BASE_URL=https://openrouter.ai/api/v1
```

نظام Auto-Fallback



المدير (ProviderManager) يحاول مع المزود الحالي أولاً، وإذا فشل ينتقل تلقائياً للمزود التالي حسب ترتيب provider_fallback.

```
config/default.json

"provider_fallback": ["ollama", "openai", "anthropic"]
```

الترتيب الافتراضي: Ollama (محلي) ← OpenAI (سحابي) ← Anthropic (سحابي). يمكنك تغيير الترتيب حسب احتياجك.

قنوات الاتصال

6

يدعم Adam Prism 25 قناة تواصل مقسمة إلى ثلاثة أنواع:

WhatsApp

```
config/default.json

"channels": {
  "whatsapp": {
    "enabled": true,
    "phone_number_id": "YOUR_PHONE_NUMBER_ID",
    "access_token": "YOUR_ACCESS_TOKEN",
    "verify_token": "YOUR_VERIFY_TOKEN"
  }
}
```

نقطة نهاية Webhook: `POST /webhook/whatsapp`

التحقق: `GET /webhook/whatsapp?hub.mode=subscribe&hub.verify_token=...`

Telegram

```
@BotFather إعداد البوت
@BotFather وابعث عن Telegram ا
أرسل: /newbot
اختر اسم: Adam Prism
اسم المستخدم: adam_prism_yourname_bot
انسخ Token
```

```
config/default.json
```

```
{
  "channels": {
    "telegram": {
      "enabled": true,
      "bot_token": "YOUR_BOT_TOKEN",
      "mode": "polling" // أو "webhook"
    }
  }
}
```

```
أو تشغيل البوت
python deploy/bot_entrpoint.py --token YOUR_TOKEN
```

Discord

```
{
  "channels": {
    "discord": {
      "enabled": true,
      "bot_token": "YOUR_DISCORD_TOKEN"
    }
  }
}
```


Discord يستخدم نظام Hybrid (Gateway + Webhook) معاً).

كل القنوات الـ 25

القناة	النوع	مفتاح الإعداد
Discord	Hybrid	DISCORD_TOKEN
Slack	Webhook	SLACK_BOT_TOKEN
Email	Polling	EMAIL_HOST EMAIL_USER
SMS (Twilio)	Webhook	TWILIO_ACCOUNT_SID
Facebook	Webhook	FACEBOOK_PAGE_ACCESS_TOKEN
Twitter	Polling	TWITTER_BEARER_TOKEN
Matrix	Polling	MATRIX_HOMESERVER
Signal	Polling	SIGNAL_CLI_PATH
Instagram	Polling	INSTAGRAM_USERNAME
LINE	Webhook	LINE_CHANNEL_ACCESS_TOKEN
Viber	Webhook	VIBER_AUTH_TOKEN
Teams	Webhook	TEAMS_WEBHOOK_URL
Google Chat	Webhook	GOOGLE_CHAT_WEBHOOK_URL

القناة	النوع	مفتاح الإعداد
IRC	Polling	IRC_SERVER IRC_NICK
XMPP	Polling	XMPP_JID
RSS	Polling	RSS_FEEDS
GitHub	Polling	GITHUB_TOKEN
Notion	Polling	NOTION_API_KEY
WeChat	Polling	WECHAT_CORP_ID
Generic Webhook	Webhook	— (اختياري)
WebChat	Embedded	chat/widget/ مدمج في

قالب تهيئة القنوات

```

annels": {
  telegram": {
    enabled": false,
    bot_token": "",
    mode": "webhook"

  whatsapp": {
    enabled": false,
    phone_number_id": "",
    access_token": "",
    verify_token": ""

  discord": {
    enabled": false,

```

```
bot_token": ""

webchat": {
  "enabled": true

feeds": {
  "enabled": false,
  "feeds": [],
  "interval": 300

github": {
  "enabled": false,
  "token": "",
  "repos": []
```

القنوات تبدأ تلقائياً عبر `ChannelManager.start_all()` عند تهيئتها. ليست بحاجة لتدوي.



نظام الذاكرة

7

نظام الذاكرة في Adam Prism متعدد الطبقات يجمع بين الذاكرة الشعاعية (Qdrant) والذاكرة المحلية مع نظام تخزين مؤقت ذكي.

الذاكرة الشعاعية — Qdrant

تخزين واسترجاع دلالي باستخدام التضمينات (Embeddings). كل نص يُحوّل إلى 768 رقم عبر نموذج embed-text ويُخزّن في Qdrant للبحث بالمعنى لا بالكلمات.

المجموعة	المحتوى	مثال
adam_knowledge	معرفة عامة	"مبدأ عدم اليقين ينص على..."
adam_conversations	محادثات	سؤال: "اشرح ميكانيكا الكم" → جواب
adam_patterns	أنماط تفكير	"المستخدم يفضل الإجابات المنهجية"
adam_reasoning_patterns	أنماط الاستدلال	"عند التحليل، ابدأ بالفرضيات"
adam_summaries	ملخصات	ملخص كتاب "تفكير سريع وبطيء"
adam_connections	روابط	"مبدأ عدم اليقين ↔ اتخاذ القرار"

```
Qdrant عبر Docker
run -d --name qdrant \
6333:6333 \
6334:6334 \
./qdrant_data:/qdrant/storage \
ant/qdrant:latest
```

ذاكرة SQLite المحلية

عند عدم توفر Qdrant، ينتقل النظام تلقائياً إلى SQLite ك fallback. لا حاجة لأي إجراء منك

عمليات الذاكرة

العملية	الوصف	API
تخزين معرفة	إضافة معرفة جديدة لقاعدة المعرفة	POST /api/knowledge/store

العملية	الوصف	API
بحث دلالي	البحث بالمعنى في كل المجموعات	<code>POST /api/memory/search</code>
تخزين محادثة	حفظ سؤال وجواب تلقائياً	تلقائي بعد كل محادثة
تخزين نمط	حفظ نمط تفكير أو سلوك	<code>POST /api/memory/pattern</code>
تخزين ملخص	حفظ ملخص مع المواضيع الرئيسية	<code>POST /api/memory/summary</code>
ربط أفكار	ربط فكرتين بنوع العلاقة	<code>POST /api/memory/connection</code>
استرجاع	بحث في كل المجموعات بالتوازي	<code>POST /api/memory/retrieve</code>

تهيئة الذاكرة

```
config/default.json

{
  "memory": {
    "grant_url": "http://localhost:6333",
    "llama_base": "http://localhost:11434",
    "embedding_model": "nomic-embed-text"

    "max_term_limit": 50,
    "text_window": 4096,
    "token_budget": 4000
  }
}
```

- خزّن المفاتيح في متغيرات بيئة أو ملف .env فقط
- لا تضع المفاتيح في الكود أبداً
- استخدم ADAM_API_KEY للمصادقة على API
- استخدم ADAM_ADMIN_KEY للعمليات المميزة (MCP، صلاحيات إدارية)
- ولّد مفاتيح قوية: `n3 -c "import secrets; print(secrets.token_urlsafe(32))"`

نظام الصلاحيات (Permission System)

يحدد صلاحيات كل أداة من الأدوات الـ 53 المدمجة:

الأداة	الحد الأقصى/جلسة	تحتاج تأكيد
shell	20	لا
file_write	30	نعم
browser_fetch	30	لا
screenshot	20	لا
memory_store	30	لا
memory_recall	50	لا

حارس الأمان (Security Guard)

يتكون من 3 طبقات حماية تعمل بدون استدعاء النموذج (سريعة وخفيفة):

1

InputGuard — فحص المدخلات قبل وصولها للنموذج. يكشف: حقن الأوامر، تسريب تعليم النظام، محاولات كسر القيود، حقن الكود.

↓

- 2

OutputGuard — فحص المخرجات قبل وصولها للمستخدم. يكشف: تسريب system prompt
بيانات شخصية (PII)، حقن كود.
- 3

ToolPermissionValidator — التحقق من صلاحيات استدعاء الأدوات: حد الاستدعاءات، المحظورة، التأكيد المطلوب.

البوابة الأخلاقية (Ethics Gate)

تقييم كل رد حسب 4 قوانين قبل الإرسال:

القانون	الوزن	المعيار
العدالة	40%	هل الرد منصف وصادق وغير متحيز؟
التعلم	30%	هل يساعد المستخدم على النمو؟
البقاء	20%	هل يحافظ على الأمان؟
الإبداع	10%	هل مبتكر ويحل المشكلة؟

المحرمات المطلقة — أي رد ينتهك هذه يُرفض فوراً: إيذاء جسدي/نفسي، انتهاك خصوصية، تزوير معلومات، التلاعب بقرارات المستخدم، إخفاء معلومات مهمة عمداً.

سجل التدقيق (Audit Logging)

كل استدعاء أداة يُسجّل مع: اسم الأداة، المعاملات، السماح/المنع، السبب، الوقت. يمكنك الوصول عبر

```
API  
api/security/audit-log?limit=50
```

```
ToolPermissionValidator
tor.get_audit_log(limit=50)
```

حماية SSRF

- فحص عناوين URL قبل طلبها لمنع الوصول للشبكة الداخلية
- تعقيم محتوى الويب غير الموثوق عبر Base64 encoding
- تحديد أقصى لحجم المحتوى (max_chars=4000)

أمان Shell

- تنفيذ الأوامر في بيئة محدودة (sandbox)
- حد أقصى 20 استدعاء shell لكل جلسة
- مراقبة الأوامر الخطيرة (os.system, subprocess.run, exec)
- تسجيل كل أمر في سجل التدقيق

الأدوات والمهارات

9

الأدوات المدمجة (53 أداة)

الأدوات	الفئة
read, write, list, find, grep, tail, copy, delete, info	ملفات
info, processes, memory, network, uptime, disk_space	نظام
status, diff, log, commit, push, pull, clone	Git

الأدوات	الفئة
search, fetch, http_get, ping, dns, whois, screenshot	ويب
calc, hash, uuid, date, base64, json, csv	أدوات
compress, decompress, zip	ضغط
pip, npm, apt	حزم
store, recall, list, todo	ذاكرة
subagent, sandbox, mcp_call, mcp_register	متقدمة

تكامّل MCP (Model Context Protocol)

Adam Prism يعمل كمضيف MCP (يقدم أدواته) وعميل MCP (يتصل بخوادم MCP خارجية) — 70- إضافية.

الوصف	أداة MCP
إرسال رسالة وreceive رد	adam_chat
تخزين ذاكرة مع بيانات وصفية	adam_memory_store
بحث في الذكريات	adam_memory_search
عرض المهارات المحملة	adam_skill_list
تحميل مهارة بالاسم	adam_skill_load
جلب محتوى صفحة ويب	adam_browser_fetch
تنفيذ أداة نظام	adam_tools_execute

الوصف	أداة MCP
حالة المحرك	adam_status

خوادم MCP الخارجية المدعومة: filesystem, Git, GitHub, Docker, Kubernetes, PostgreSQL, Playwright, Slack, Notion, Jira, Figma, Sentry, Cloudflare وغيرها.

إعداد MCP مع Claude Desktop

```
library/Application Support/Claude/claude_desktop_config.json

"servers": {
  "adam-prism": {
    "command": "adam-prism",
    "args": ["mcp"],
    "env": {
      "OLLAMA_BASE": "http://localhost:11434"
    }
  }
}
```

إعداد خوادم MCP مخصصة

```
fig/mcp_servers.json

"servers": [
  {
    "name": "weather-api",
    "transport": "stdio",
    "command": "python",
    "args": ["-m", "weather_mcp_server"],
    "env": { "API_KEY": "your-key" }
  }
]
```

نظام المهارات (Skills)

المهارات حزم معرفية قابلة لإعادة الاستخدام تعلّم آدم كيفية التعامل مع مهمة محددة. ليست إضافات معرفة مركزة.

code-review

مراجعة الكود واكتشاف المشاكل

git-commit

كتابة رسائل commit تقليدية

debug

تشخيص الأخطاء وإصلاحها

explain-code

شرح الكود خطوة بخطوة

write-test

كتابة اختبارات وحدة شاملة

إدارة المهارات CLI

```
kill list --installed      # عرض المهارات المثبتة
kill install git-commit    # تثبيت مهارة من السوق
kill search "docker"       # بحث في السوق
kill update --all          # تحديث كل المهارات
kill rate git-commit 5     # تقييم مهارة
```

نظام الإضافات (Plugins)

نظام إضافات مبني على الـ hooks يسمح باعتراض وتعديل سلوك المحرك في نقاط محددة:

1 before_generate() — تعديل الرسالة/السياق قبل توليد الرد

2 after_generate() — تعديل الرد بعد التوليد

3 before_tool() — اعتراض تنفيذ الأداة (أو منعها)

4 after_tool() — تعديل نتيجة الأداة

مثال: إنشاء إضافة

```
from langchain.plugins import AdamPlugin

class MyPlugin(AdamPlugin):
    name = "my-plugin"
    version = "1.0.0"
    description = "Does something useful"

    async def on_load(self, engine):
        self.engine = engine

    async def before_generate(self, message, context):
        context["extra_info"] = "added by plugin"
        return {"message": message, "context": context}

    async def after_generate(self, message, response):
        return response + "\n\n- Powered by MyPlugin"
```

الوكلاء الفرعيين

10

إنشاء وكيل فرعي (Spawning)

يمكن لآدم إنشاء وكلاء فرعيين متخصصين للعمل على مهام محددة. كل وكيل له جلسة منفصلة وأدوات

```
API
api/subagents/spawn

name": "research-agent",
config": {
  mode": "researcher",
  tools_enabled": ["search", "browser_fetch", "memory_store"]
}
```

القيود:

- ADAM_MAX_SUBAGENTS) الحد الأقصى: 10 وكلاء فرع
- الأسماء المحظورة: admin, root, system, adam, sudo
- لا يمكن إنشاء وكيل بنفس الاسم
- تنظيف تلقائي للوكلاء الخاملين بعد 30 يوم

إدارة الفرق (Team Management)

يمكن تنظيم الوكلاء في فرق تعمل معاً:

```
إشياء
api/subagents/teams

name": "analysis-team",
members": ["research-agent", "analyst-agent", "writer-agent"],
mode": "parallel" // أو "sequential"
```

التنفيذ المتوالي vs المتوازي

الوضع	الوصف	متى تستخدمه
Sequential	الوكلاء يعملون واحداً تلو الآخر، كل واحد يبني على نتيجة السابق	المهام التي تحتاج ترتيب
Parallel	الوكلاء يعملون في نفس الوقت	المهام المستقلة التي لا تعتمد بعضها

إدارة الوكلاء

```
عرض كل الوكلاء
GET /api/subagents/list

محادثة وكيل
POST /api/subagents/{id}/chat
{ "message": "حلل هذه البيانات" }

حذف وكيل
DELETE /api/subagents/{id}

حذف كل الوكلاء
DELETE /api/subagents/all
```

خط الأنابيب والجدولة

11

مراقبة خط الأنابيب (Pipeline Monitoring)

يمكن مراقبة أداء النظام عبر Prometheus و Grafana المدمجين:

```
prometheus
localhost:9090
```

```
ana
localhost:3001
البيانات : admin / ${GRAFANA_ADMIN_PASSWORD}
```

لوحة Grafana المضمنة تعرض: معدل الطلبات، وقت الاستجابة، استخدام الأدوات، حالات الأمان.

نظام الجدولة (Scheduler)

يدعم جدولة المهام بثلاثة أنواع:

Interval فاصل

تنفيذ مهمة كل فترة زمنية محددة (مثل: كل 5

Cron دوري

تنفيذ مهمة بجدول زمني محدد (مثل: كل يوم الساعة 8 صباحاً)

One-time مرة واحدة

تنفيذ مهمة في وقت محدد مرة واحدة فقط

```
API
api/scheduler/cron

{
  "id": "daily-summary",
  "cron_expr": "0 8 * * *",
  "description": "ملخص يومي"
}
```

```
api/scheduler/interval

{
  "id": "health-check",
  "interval_seconds": 300,
  "description": "فحص صحي"
}
```

```
api/scheduler/once

id": "backup-now",
at": "2026-03-05T02:00:00",
": "نسخ احتياطي"
```

```
إدارة ا
api/scheduler/list      # عرض كل المهام
api/scheduler/{job_id}  # تفاصيل مهمة
/api/scheduler/{job_id} # حذف مهمة
```

أمثلة على Cron

الوصف	التعبير
كل يوم الساعة 8:00 صباحاً	* * * 8 0
كل ساعتين	* * * 2/* 0
كل اثنين الساعة 12:30 بعد منتصف الليل	1 * * 0 30
أول يوم من كل شهر	* * 1 0 0

صيغة Cron: دقيقة ساعة يوم شهر يوم_الأسبوع — 5 أجزاء مفصولة بمسافات.



النسخ الاحتياطي والاستعادة

12

نسخ البيانات احتياطياً

نسخ بيان Qdrant

```
exec adam-qdrant curl -X POST http://localhost:6333/snapshots
```

داخل الحاوية /qdrant/snapshots النسخة تُحفظ

أو نسخ المجلد م

```
qdrant_data/ backup/qdrant_$(date +%Y%m%d)/
```

(محدثات وسجلات) API نسخ بيان

```
data/ backup/data_$(date +%Y%m%d)/
```

```
logs/ backup/logs_$(date +%Y%m%d)/
```

نسخ Ollama نما

```
/.ollama/ backup/ollama_$(date +%Y%m%d)/
```

نسخ شامل بسكريبت و

```
tf adam-backup-$(date +%Y%m%d).tar.gz \
```

```
qdrant_data/ \
```

```
ta/ \
```

```
logs/ \
```

```
config/ \
```

```
tebook/
```

نسخ الإعدادات احتياطياً

نسخ ملفات ا

```
fig/default.json backup/config_$(date +%Y%m%d).json
```

```
/ backup/env_$(date +%Y%m%d)
```

نسخ مهاراتك ال

```
/.adam/skills/ backup/skills_$(date +%Y%m%d)/
```

نسخ إضاف

```
plugins/ backup/plugins_$(date +%Y%m%d)/
```

إجراءات الاستعادة

```

استعادة Qdrant
أوقف الـ
stop adam-qdrant

استعادة البيانات
backup/qdrant_YYYYMMDD/ qdrant_data/

أعد تشغيل الـ
start adam-qdrant

استعادة API
backup/data_YYYYMMDD/ data/
backup/logs_YYYYMMDD/ logs/

استعادة الإعدادات
kup/config_YYYYMMDD.json config/default.json
kup/env_YYYYMMDD .env

استعادة من أرشيف شا
tf adam-backup-YYYYMMDD.tar.gz

إعادة تشغيل الـ
oy && docker compose restart

```

أوقف كل الخدمات قبل الاستعادة لتجنب تلف البيانات. استخدم `bash stop.sh` أو `docker compose down`.



حل المشاكل الشائعة

13

Ollama لا يعمل

الخطأ: ConnectionRefusedError: [Errno 111] Connection refused: على

11434



فحص Ollama

```
list
```

إعادة ال

```
systemctl restart ollama
```

فحص ا

```
journalctl -u ollama -n 50
```

تشغيل

```
serve
```

فحص ا

```
http://localhost:11434/api/tags
```

تعارض المنافذ

الخطأ: Address already in use



معرفة من يشغل ا

```
:8000
```

```
ps | grep 8000
```

قتل ال

```
$ (lsof -ti :8000)
```

تغيير ا

```
main.py --port 8001
```

المنفذ	الخدمة	الحل
3000	Web UI	تغيير عبر <code>npm run dev -- -p 3001</code>
6333	Qdrant	تغيير عبر <code>p 6334:6333-</code>
8000	API	تغيير عبر <code>port 8001-</code>
11434	Ollama	تغيير عبر <code>OLLAMA_HOST</code>

مشاكل اتصال Qdrant

الخطأ: Qdrant connection refused



```
docker
ps | grep qdrant

إعادة تشغيل الـ
restart adam-qdrant

فحص |
logs adam-qdrant --tail 50

فحص |
http://localhost:6333/collections
```

إذا لم يكن Qdrant متاحاً، ينتقل النظام تلقائياً إلى SQLite. لا حاجة لإجراء فوري.



أخطاء الاستيراد (ImportError)

```
: No module named adam
install -e .

: No module named httpx
install -r requirements.txt

: Playwright not found
 playwright install firefox
```

أخطاء الصلاحيات

```
: Permission denied على ملفات
chown -R $USER:$USER data/ logs/ notebook/

: Docker permission denied
usermod -aG docker $USER
سجل خروج ودخول
```

مشاكل الذاكرة

```
: CUDA out of memory
استخدم نموذج أصغر
pull gemma3:4b # بدلاً من 12B

قلل السيل
config/default.json:
"text_window": 2048 }

أوقف العمليات الأخرى
nvidia-smi # فحص استخدام GPU
```

فحص صحي شامل

```
فحص كل ال
http://localhost:8000/api/engine/health # API
http://localhost:11434/api/tags # Ollama
http://localhost:6333/collections # Qdrant
```

إرسال رسالة تج

```
{ POST http://localhost:8000/api/chat \
Content-Type: application/json" \
Authorization: Bearer $ADAM_API_KEY" \
{"message": "مرحباً يا آدم", "session_id": "test"}'
```

تشغيل الاخت

```
tests/ -k "not slow"
```

تطبيقات الديسكتوب والموبايل

14

تطبيق سطح المكتب (Electron)

تطبيق مكتبي مبني على Electron 33 مع نموذج أمان preload.js.

```
ال
ntend/desktop-app
stall

التشغيل (وضع الت
art

بناء ل
a build:linux # → AppImage

بناء macOS
a build:mac # → DMG

بناء Windows
```

```
build:win # → NSIS installer
```

بناء لكل ال

```
build:all
```

الخاصية	التفاصيل
المعرف	<code>com.adam-prism.desktop</code>
الإطار	Electron 33
نموذج الأمان	preload.js (contextBridge)
المنصات	Linux (AppImage), macOS (DMG), Windows (NSIS)

إضافة VS Code

```
ال  
tend/vscode-extension
```

```
stall
```

```
compile
```

الأوامر ال

alt+a → فتح المحادثة

alt+e → شرح الكود

prism.chat → محادثة

prism.explainCode → شرح كود

prism.reviewCode → مراجعة كود

prism.debugCode → تشخيص خطأ

prism.writeTest → كتابة اختبار

تطبيق Android

تطبيق مبني على Flutter مع دعم كامل للعربية و RTL:

```
المتن
Flutter SDK 3.x
Android Studio / SDK

الخطوات
flutter doctor
flutter pub get

تشغيل على جهاز
flutter run

تصدير APK
flutter build apk --release

تصدير App Bundle (لنشر)
flutter build appbundle --release
```

الخاصية	التفاصيل
الإطار	Flutter
اللغات	العربية + الإنجليزية (l10n)
الاتصال	SSE + REST API + WebSocket
الميزات	محادثة، ذاكرة، مهارات، أدوات، إعدادات، تعلم مبدئي
التخزين	محلي (Hive) مع مزامنة اختيارية
الحد الأدنى	Android 5.0 (API 21)

تطبيق iOS


```
المتطلبات:  
- macOS مع Xcode 15+  
- Flutter SDK 3.x  
- Apple Developer Account (للنشر)  
  
الخطوات:  
1. تثبيت Flutter SDK  
$ flutter pub get  
  
2. تشغيل على جهاز  
$ flutter run  
  
3. بناء لـ iOS  
$ flutter build ios --release  
  
4. للنشر في Xcode  
افتح Runner.xcworkspace
```

تطبيق iOS يحتاج macOS مع Xcode. للتطوير فقط، يمكنك استخدام المحاكى (simulator) 

الوصول من أجهزة متعددة عبر Tailscale

```
1. تثبيت Tailscale على خادم (Linux)  
$ curl -fsSL https://tailscale.com/install.sh | sh  
$ tailscale up  
  
2. الحصول على IP  
$ tailscale ip -4  
مثال: 10.4.0.1  
  
3. تثبيت Tailscale على iPad  
تحميل Tailscale من App Store / Google Play  
سجل دخول بنفسك  
  
4. افتح الـ
```

```
//100.x.x.x:3000 → واجهة الويب  
//100.x.x.x:8000 → API مباشرة
```

آدم بريزم — التوأم الرقمي الذي يفهمك ويتعلم منك وينمو معك

Born in Egypt · Built for the World · Free forever · Apache 2.0

Adam Prism v1.0.0 — دليل التشغيل التفصيلي

Apache 2.0 License · [البلاغات](#) · [GitHub](#)